

INFORME PROYECTO FINAL

Jhork Acevedo, isaac Bustamante, Nicolás García

Universidad del Quindío

Armenia, Quindío

jhorkw.acevedoc@uqvirtual.edu.co

nicolas.garciap@uqvirtual.edu.co

isaac.bustamanteo@uqvirtual.edu.co

Abstract— El presente informe describe el proceso de desarrollo y justificación del proyecto final del curso de programación, a través de la concepción, diseño e implementación, usando paradigmas de programación, como, programación orientada a objetos, programación de interfaces gráficas mediante la librería custom tkinter y manejo de archivos.

I. INTRODUCCIÓN

En el contexto de la gestión laboral en empresas, el registro y seguimiento de documentos laborales, explícitamente de contratos laborales, se acostumbra a manejar cantidades considerables de datos, tales como salario pactado, condiciones, fechas de inicio y finalización.

Por lo tanto, al no asegurar un sistema óptimo a una entidad, esta corre el riesgo de no renovar tales documentos, al no pago de garantías y seguros legales, quedando vulnerable a cualquier riesgo legal.

II. CONCEPCIÓN

Por dicha razón, se concibe un sistema en software, el cual deberá, mover automáticamente datos registrados y nuevos, restringiendo la cantidad de operaciones dependiendo del rol de la persona en la entidad, además, el usuario deberá usar el sistema de la manera más simple posible, evitando errores al entrar datos.

Además el sistema debe ser simplificado de manera visual mediante una interfaz gráfica dinámica y amigable con el usuario.

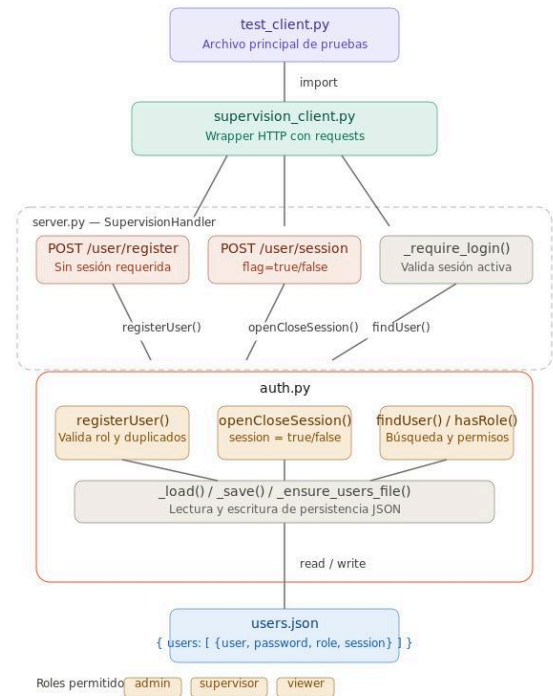
Por otro lado, el sistema deberá validar automáticamente, los datos registrados por el usuario, como el correo electrónico, fechas de inicio y fin entre otros, siendo un sistema efectivo y óptimo, el cual también sea interconectable con las personas de la entidad.

II.DISEÑO

A. Diseño de Back-end

Para cumplir los requerimientos del sistema, se opta por la implementación de un servidor ya provisto por el profesor, el cual no es modificado para cumplir con la intercomunicación del sistema de acuerdo a la entidad.

Además para validar las funcionalidades del usuario restringidos mediante su rol, se propone el siguiente código de funciones, el cual se usará más adelante en un archivo principal



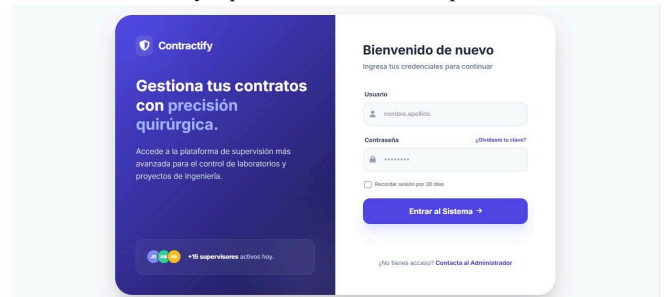
Figura[1].

Se plantea una estructura de código modular funcional que será usada por el servidor posteriormente, dicha estructura se le es asignada el nombre “auth.py” y “supervision.py”, las cuales ya fueron especificadas en el informe anterior [].

B. Diseño de Front-end

1. Diseño inicio de sesión

Para cumplir con el requerimiento de simplificación del sistema mediante un diseño gráfico, se implementa el uso de la librería custom tkinter, ya que el diseño de mockup es moderno.

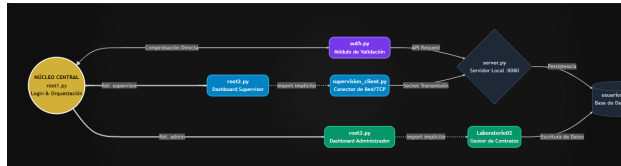


Figura[2]

Debido a este diseño, se implementa dicha librería, en la cual

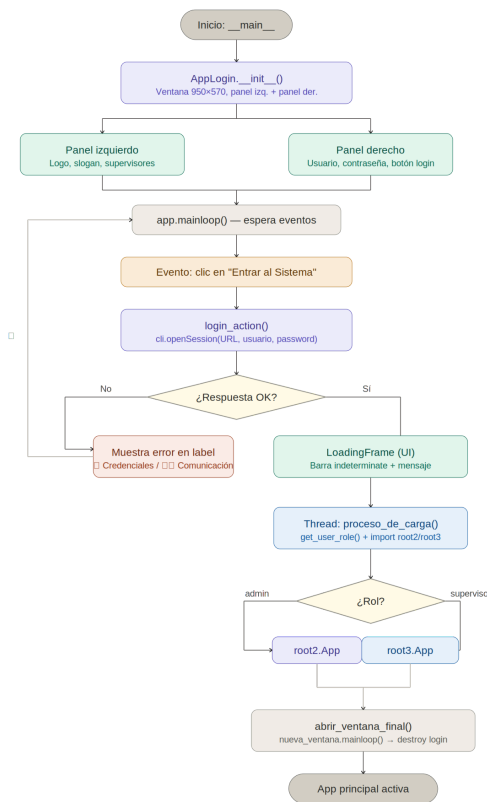
para el flujo del programa se proponen 3 rutas, una para el inicio de sesión, otra para el cuadro de operaciones del supervisor y por último otra para el cuadro de operaciones del administrador.

Cada una de estas rutas contienen cada uno de los archivos en el que se llama al back end desarrollado, tal como se muestra en la siguiente figura:



Figura[3]

Por tanto, mediante el paradigma de programación orientada a objetos se crea la interfaz siguiendo el siguiente flujo para [root1.py](#) o tambien la ruta de inicio de sesion:



Figura[4]

Cabe aclarar que la lógica de creación de labels, botones, canvas y funciones se hace de la misma manera que en tkinter. Un ejemplo es la creación de botones, usados en esta ruta ([root1.py](#)), según [2], para cambiar el color de fondo en un botón puede hacerse de 3 maneras, mediante la clave específica, mediante clave hexadecimal o como tupla de claves:

```
button = customtkinter.CTkButton(rootTk,
fg_color="red") # single color name
button = customtkinter.CTkButton(rootTk,
fg_color="#FF0000") # single hex string
button = customtkinter.CTkButton(rootTk,
fg_color=("#DB3E39", "#821D1A")) # tuple
color
```

Figura[5]

También, pueden ser personalizados mediante argumentos específicos [2]:

```
customtkinter.set_default_color_theme("dark-
blue") # Themes: "blue" (standard),
"green", "dark-blue
customtkinter.set_default_color_theme("path/
to/your/custom_theme.json")
```

Figura[6]

Al aceptar y definir parámetros más flexibles, permite crear diseños más modernos, además de contar con más “Widgets” que caracterizan un diseño modesto. Por ende, se opta por la implementación de dicha librería.

Por otro lado, también es importante sustentar el diseño de la pantalla de carga como animación de espera mientras se crea la nueva ventana según el flujo figura[4], ya que para la creación de la animación de carga, custom tkinter ya cuenta con un objeto llamado Progress Bar:

```
self.progress =
ctk.CTkProgressBar(self,
orientation="horizontal",
mode="indeterminate", width=300,
progress_color="#4d44ef")
self.progress.grid(row=1,
column=0, pady=(0, 100))
self.progress.start()
```

Figura[7]

Tal como se aprecia en esta línea de código de [root1.py](#) mediante los argumentos de dicho objeto, se determinan la orientación en horizontal y su progreso de forma indeterminada.

El siguiente paso en el desarrollo del proyecto final se detalla la comunicación entre el servidor y cada ruta.

Mediante NGrok, se consiguió un dominio de forma gratuita [4] para tener intercomunicación entre usuarios de una misma entidad. Es necesario tener en cuenta que dicho dominio es estático y no personalizado[3], lo que quiere decir que su uso no es de forma comercial o para producción, sino para desarrollo personal.

Por tanto, en la URL de cada root se copia y pega la siguiente ruta de dominio:

```
URL_SERVER =
"https://fifty-stardust-backlog.ngrok-
free.dev"
```

2. Diseño de pantalla principal del supervisor

Para la gestión de contratos por parte del supervisor, se diseñó una interfaz principal dividida en dos secciones: una barra lateral (sidebar) que contiene las opciones de navegación y el perfil del usuario, y un área central desplegable donde se despliega el contenido dinámico. Figura[8]:

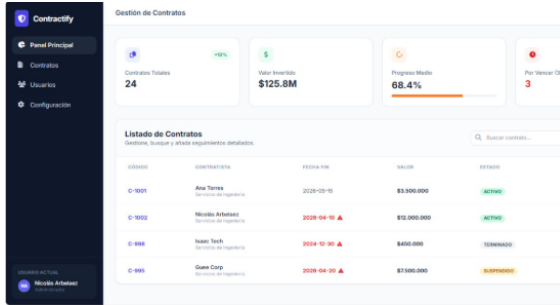


Figura [8]. Pantalla principal del supervisor

La barra lateral se construye mediante instancias de CTKFrame y CTKButton. El menú de contratos incluye un submenú desplegable que permite al supervisor acceder rápidamente a las funciones de listar contratos, buscar contrato, añadir seguimiento y ver seguimientos. La implementación del submenú se realiza mediante un CTKFrame contenedor cuya visibilidad se alterna con el método pack_forget() y pack() al presionar el botón principal [2] Figura[9].

```
def toggle_submenu(self):
    if self.submenu_visible:
        self.submenu_frame.pack_forget()
        self.btn_contratos.configure(text="■ Contratos ▶")
    else:
        self.submenu_frame.pack(fill="x", after=self.btn_contratos)
        self.btn_contratos.configure(text="■ Contratos ▼")
        self.submenu_visible = not self.submenu_visible
```

Figura[9]. Implementación del submenú desplegable.

A. Estructura del área principal desplazable

Para el área central de la interfaz, se utiliza CTKScrollableFrame, un widget de CustomTkinter que permite desplazamiento vertical cuando el contenido excede la altura disponible [2]. Esto es especialmente útil en la tabla de contratos, que puede crecer dinámicamente a medida que se registran nuevos contratos. Figura [10].

```
self.main_container = ctk.CTKScrollableFrame(self, fg_color=COLOR_BG_MAIN, corner_radius=0)
self.main_container.grid(row=0, column=1, sticky="nsew", padx=5, pady=5)
```

Figura[10]. Área principal desplazable.

B. Construcción dinámica de la tabla de contratos

A diferencia de tkinter.ttk.Treeview, CustomTkinter no posee un widget de tabla nativo [2]. Por esta razón, la tabla se construye manualmente mediante frames y labels organizados en grid. Figura [11].

```
for row, c in enumerate(contratos):
    row_frame = ctk.CTKFrame(self.rows_frame, fg_color="transparent", height=50)
    row_frame.pack(fill="x", pady=2)
    ctk.CTKLabel(row_frame, text=c.get("number", ""), width=120).grid(row=0, column=0)
    ctk.CTKLabel(row_frame, text=c.get("contractor", "")[:30], width=150).grid(row=0, column=1)
    btn_ver = ctk.CTKButton(row_frame, text="Ver", command=lambda num=c.get("number"): self.ver_detalle_contrato(num))
    btn_ver.grid(row=0, column=5)
```

Figura[11]. Generación dinámica de filas en la tabla de contratos

C. Tarjetas de estadísticas y barra de progreso

En el área central se encuentran cuatro tarjetas de estadísticas, implementadas con CTKFrame, que muestran métricas clave como el total de contratos, el valor invertido, el progreso medio y los contratos por vencer en 30 días. Estos datos son obtenidos dinámicamente desde el servidor mediante la función stats() del módulo supervision_client. La tarjeta de "Progreso Medio" incorpora una CTKProgressBar que refleja visualmente el avance promedio [2], Figura [12].

```
self.progress_bar = ctk.CTKProgressBar(self.card_progreso, width=150, height=6, progress_color="#10b981")
self.progress_bar.pack(anchor="w", padx=15, pady=(5, 15))
self.progress_bar.set(avg_progress / 100)
```

Figura[12]. Tarjetas de estadísticas y barra de progreso

III. IMPLEMENTACIÓN FULL-STACK:

La comunicación entre la interfaz gráfica del supervisor y el servidor se realiza a través del módulo supervision_client.py, el cual encapsula las peticiones HTTP necesarias para consumir los endpoints expuestos por server.py. A continuación, se detalla el flujo de cada operación disponible en la pantalla del supervisor.

A. Listado de contratos.

Cuando el supervisor selecciona la opción "Listar Contratos". Figura[13]

```
respuesta = cli.listContracts(URL, self.usuario, self.password)
data = json.loads(respuesta)
contratos = data.get("contracts", [])
self.mostrar_contratos_en_tabla(contratos)
```

Figura [13]. Acceder a lista de contratos.

B. Búsqueda de contrato

Esta función solicita al usuario un número de contrato mediante un cuadro de diálogo (simpledialog.askstring). Figura[14]

```
respuesta = cli.searchContract(URL, self.usuario, self.password, numero)
```

Figura [14]. Implementación de búsqueda de contratos.

El servidor procesa la petición GET al endpoint /search y retorna el contrato completo si existe. La interfaz muestra el resultado en la misma tabla, reemplazando el listado completo por el contrato encontrado.

C. Registro de contrato.

Esta operación abre una ventana emergente (CTkToplevel) con un formulario que captura todos los campos necesarios: número, contratista, objeto, fechas, valor, supervisor, estado y correo electrónico. Al enviar, se ejecuta en figura[15].

```
respuesta = cli.registerContract(URL, self.usuario, self.password, ...)
```

Figura [15]. Implementación de registro de contratos.

El cliente envía una petición POST al endpoint /register con los datos del formulario. El servidor valida el correo, las fechas, el valor y la unicidad del número de contrato. Si la validación es exitosa, el contrato se guarda en db.json utilizando el módulo json de la biblioteca estándar de Python [1] y se retorna un mensaje de éxito; de lo contrario, se retorna un error específico que se muestra al usuario mediante messagebox.

D. Añadir seguimiento

Esta función permite al supervisor registrar el avance de un contrato. Solicita el número de contrato y luego abre un formulario para ingresar fecha, descripción, porcentaje de progreso (0-100) y observaciones. La petición se envía a través de la figura [16].

```
respuesta = cli.addTracking(URL, self.usuario, self.password, numero, fecha, desc, progreso, obs)
```

Figura [16]. Implementación de añadir seguimiento.

El servidor procesa la petición POST al endpoint /tracking/add, verifica que el contrato exista y que el progreso esté en el rango válido, y luego agrega el seguimiento al arreglo trackings del contrato correspondiente en db.json.

E. Ver seguimientos

Esta opción solicita un número de contrato y consulta los seguimientos asociados mediante la figura[17].

```
respuesta = cli.listTrackings(URL, self.usuario, self.password, numero)
```

Figura [17]. Implementación de ver seguimiento.

El servidor atiende la petición GET al endpoint /tracking/list y retorna la lista de seguimientos del contrato [1]. La interfaz presenta esta información en una ventana emergente con un área de texto desplazable construida con Text y Scrollbar de tkinter.

F. Actualización de estadísticas.

Las tarjetas superiores no son estáticas. Cada vez que el supervisor carga el panel principal o registra un nuevo contrato, se invoca la función cargar_stats(), que consulta al servidor el endpoint /stats y actualiza los valores mostrados. Figura [18].

```
def cargar_stats(self):
    respuesta = cli.stats(URL, self.usuario, self.password)
    stats = json.loads(respuesta)
    self.card_0_label.configure(text=str(stats.get("total_contracts", 0)))
    self.card_1_label.configure(text=f"${stats.get('total_value', 0):,.0f}")
```

Figura [18]. Implementación de Actualización de estadística

G.Cierre de sesión e importación dinámica

Esta petición POST al endpoint /user/session con el parámetro flag=false actualiza el estado de sesión del usuario en users.json, lo que impide que las credenciales puedan ser reutilizadas sin un nuevo inicio de sesión [1].

Para volver a la pantalla de login, se utiliza importación dinámica dentro del método cerrar_session(), evitando así la importación circular que ocurriría si root1 y root2 se importaran mutuamente al inicio del archivo. Figura[19].

```
def cerrar_session(self):
    self.destroy()
    import root1
    login = root1.AppLogin()
    login.mainloop()
```

s. Figura[19]. Implementación del cierre de sesión

INTERFAZ DEL ADMINISTRADOR (Main Admin):

Esta etapa del proyecto final cuenta con la funcionalidad de registrar contratos, visualizarlos, almacenarlos, registrar los nuevos usuarios y la consulta de estadísticas generales y la exportación de información hacia archivos CSV. Además, el sistema permite gestionar sesiones de usuario, diferenciando los roles y permisos dentro del programa..

para esto primero se exportaron la siguientes librería:

import customtkinter as ctk: Se encarga de importar la librería CustomTkinter, utilizada para crear interfaces gráficas modernas con botones, ventanas y componentes visuales mejorados respecto al Tkinter tradicional, incluyendo soporte para modo oscuro y claro.

from tkinter import messagebox: Importa el módulo messagebox de Tkinter, utilizado para mostrar ventanas emergentes de información, advertencia o error dentro de la aplicación

import supervision_client as cli: Importa el módulo encargado de realizar la comunicación entre la aplicación y el servidor mediante peticiones HTTP para enviar y consultar información del sistema.

Este es el cliente de comunicación. Es un script externo encargado de hacer las peticiones HTTP/API al servidor para traer o enviar los datos reales.

Tras haber importado las librerías necesarias, se definió la paleta de colores que maneja la interfaz gráfica del sistema. Cada color fue almacenado en variables para facilitar su reutilización y mantener una apariencia visual consistente en todo el programa, como se aprecia en la siguiente figura [19].

```

5
6 COLOR_BG_SIDEBAR = "#121726"
7 COLOR_BG_MAIN = "#f3f4f6"
8 COLOR_CARD = "white"
9 COLOR_ACCENT = "#4f46e5"
10 COLOR_ACCENT_HOVER = "#6366f1"
11 COLOR_TEXT_MAIN = "#111827"
12 COLOR_TEXT_SECONDARY = "#6b7280"
13

```

figura[19] variable con su respectiva paleta de colores.

Estas variables permiten centralizar la configuración visual de la interfaz, facilitando futuros cambios de diseño sin necesidad de modificar múltiples líneas de código.

Posteriormente, se procedió a crear una clase hija denominada App, la cual hereda de la clase principal proporcionada por la librería CustomTkinter importada como ctk. La herencia permite que la clase App adquiera todas las funcionalidades y características de una ventana moderna de CustomTkinter, tales como:

- manejo de ventanas,
- renderizado de componentes gráficos,
- configuración visual,
- eventos de interacción,
- y administración de widgets.

Dicha clase contiene un constructor, encargado de inicializar la interfaz principal del sistema. El método constructor recibe como parámetros el usuario y la contraseña del usuario autenticado en el sistema y los guarda en variable. Y se inicializa la clase padre tk. Como se muestra en la siguiente figura[20]

```

Windsurf: Refactor | Explain
6 class App(ctk.CTk):
7     def __init__(self, usuario, password):
8         super().__init__()
9

```

figura[20] Se crea la clase App.

Posteriormente, se configuraron las propiedades principales de la ventana de la aplicación, estableciendo el título "Contractify - Supervisor", una geometría de 1200 x 750 píxeles y el color de fondo principal de la interfaz mediante la paleta de colores previamente definida. Adicionalmente, se configuró el sistema de distribución visual utilizando el gestor **grid**, permitiendo organizar dinámicamente la interfaz en un panel lateral de navegación y un contenedor principal de contenido, como se aprecia en la siguiente figura [19].

```

self.title("Contractify - Supervisor")
self.geometry("1200x750")
self.configure(fg_color=COLOR_BG_MAIN)

self.grid_columnconfigure(1, weight=1)
self.grid_rowconfigure(0, weight=1)

```

figura[21].propiedades basicas de la ventana.

Se procedió a crear el panel lateral de navegación (sidebar) utilizando un componente CTkFrame, al cual se le definió un ancho fijo de 260 píxeles, bordes sin redondeo y un color de fondo correspondiente a la paleta de colores previamente establecida. Seguidamente, el contenedor fue ubicado en la primera columna de la ventana mediante el gestor grid, permitiendo que ocupara completamente el espacio asignado dentro de la interfaz. Además, se desactivó la propagación automática de tamaño con el fin de conservar las dimensiones definidas inicialmente.

También se añadió un componente CTkLabel que funciona como encabezado visual del panel lateral, mostrando el nombre del sistema "Contractify" junto con un ícono representativo, utilizando una tipografía personalizada, como se aprecia en la siguiente figura [22].

```

356
357
358
359 self.sidebar = ctk.CTkFrame(self, width=260, corner_radius=0, fg_color=COLOR_BG_SIDEBAR)
360 self.sidebar.grid(row=0, column=0, sticky="nsew")
361 self.sidebar.grid_propagate(False)
362
363 # Logo
364 ctk.CTkLabel(self.sidebar, text="Contractify", font=ctk.CTkFont(size=22, weight="bold"), text_color="white").pack(padx=(30, 20), pady=20, anchor="w")
365

```

figura[22]- panel lateral y su respectivos componentes.

En esta sección se implementó el menú principal de navegación de la aplicación mediante una serie de botones ubicados en el panel lateral (sidebar). Cada botón fue generado utilizando el método reutilizable `crear_boton_menu`, permitiendo mantener uniformidad visual y facilitar el acceso a las distintas funcionalidades del sistema, como se aprecia en la siguiente figura [23].

```

# Menús
self.crear_boton_menu("Panel Principal", self.cargar_dashboard)
self.crear_boton_menu("Contratos", self.listar_contratos)
self.crear_boton_menu("Registrar Contrato", self.registrar_contrato)
self.crear_boton_menu("Registrar Usuario", self.registrar_usuario)
self.crear_boton_menu("Estadísticas", self.ver_estadisticas)
self.crear_boton_menu("Exportar CSV", self.exportar_csv)

```

figura[23] botones interactivos

Posteriormente, se incorporó un separador visual dentro del panel lateral utilizando un CTkFrame, con el fin de organizar de manera más clara las opciones del menú y la información del usuario autenticado, como se aprecia en la siguiente figura [1].

```

ctk.CTkFrame(self.sidebar, height=1, fg_color="#2c344d").pack(fill="x", padx=15, pady=20)

```


figura[24]. separador visual.

También se implementó el botón “Cerrar Sesión” mediante el componente CtkButton, el cual permite finalizar la sesión actual y regresar a la ventana de autenticación. Posteriormente, se creó el contenedor principal de contenido utilizando un CtkScrollableFrame, encargado de almacenar dinámicamente todas las vistas del sistema. Además, se añadió un encabezado principal denominado header_label y un contenedor content_frame, donde se cargan las diferentes funcionalidades de la aplicación. Finalmente, se ejecuta el método cargar_dashboard(), encargado de mostrar el panel principal al iniciar el sistema, como se aprecia en la siguiente figura [25].

```
359 # Botón cerrar sesión
360 self.btn_logout = ctk.CtkButton(self.sidebar, text=" Cerrar Sesión", fg_color="transparent",
361                                anchor="w", font=ctk.CtkFont(size=13), height=35,
362                                command=self.cerrar_sesion)
363 self.btn_logout.pack(fill="x", padx=15, pady=(20, 10))
364
365 # --- CONTENIDO PRINCIPAL ---
366 self.main_container = ctk.CtkScrollableFrame(self, fg_color=COLOR_BG_MAIN, corner_radius=0)
367 self.main_container.grid(row=0, column=1, sticky="nsw", padx=5, pady=5)
368
369 self.header_label = ctk.CtkLabel(self.main_container, text="Panel de Administración",
370                                text_color=COLOR_TEXT_MAIN, font=ctk.CtkFont(size=28, weight="bold"))
371 self.header_label.pack(anchor="w", padx=30, pady=(20, 10))
372
373 self.content_frame = ctk.CtkFrame(self.main_container, fg_color="transparent")
374 self.content_frame.pack(fill="both", expand=True, padx=30, pady=10)
375
376 self.cargar_dashboard()
377
378
```

figura[25] creacion del boton cerrar seccion y contenedor principal

Ahora se explicara las funciones que son relevantes en el programa.

se desarrolló la función registrar_contrato, encargada de permitir el almacenamiento de nuevos contratos dentro del sistema mediante una ventana emergente con distintos campos de información. Seguidamente, el sistema verifica que los datos ingresados sean válidos y realiza el envío de la información al servidor utilizando cli.registerContract. Finalmente, se muestra un mensaje de confirmación y se actualiza automáticamente la interfaz, como se aprecia en la siguiente figura [26].

```
Window Refactor | Explain | Generate Docstring | X
def registrar_contrato(self):
    dialog = ctk.CtkToplevel(self)
    dialog.title("Registrar Contrato")
    dialog.geometry("500x650")
    dialog.configure(fg_color=COLOR_BG_MAIN)
```

figura[26] funcion registrar contratos

Desarrolló la función ver_estadisticas, cuya finalidad consiste en mostrar información estadística relacionada con los contratos registrados. Para ello, el sistema realiza una petición mediante cli.stats y posteriormente organiza los datos dentro de la interfaz gráfica, como se aprecia en la siguiente figura [27]-

```
def ver_estadisticas(self):
    self.limpiar_content()
    self.header_label.configure(text=" Estadísticas Detalladas")
```

figura[27]. función ver estadísticas

Por ultimo se crearon las dos ultimas funciones que son exportar_csv y cerrar_sesion. La función exportar_csv permite exportar la información almacenada en el sistema hacia archivos CSV mediante cli.exportCsv, mostrando mensajes de confirmación o error según el resultado del proceso. Por otra parte, la función cerrar_sesion se encarga de finalizar la sesión activa del usuario y retornar nuevamente a la ventana de autenticación del sistema, como se aprecia en la siguiente figura [28].

```
def exportar_csv(self):
    try:
        respuesta = cli.exportCsv(URL, self.usuario, self.password)
        res = json.loads(respuesta)
        if "error" in res:
            messagebox.showerror("Error", res["error"])
        else:
            messagebox.showinfo("Éxito", "Datos exportados a contracts.csv y trackings.csv")
    except Exception as e:
        messagebox.showerror("Error", f"Error al exportar: {e}")

Window Refactor | Explain | Generate Docstring | X
def cerrar_sesion(self):
    try:
        cli.closeSession(URL, self.usuario, self.password)
    except:
        pass
    self.destroy()
    import root1
    login = root1.AppLogin()
```

figura[28] función para exportar csv

A continuación en la figura[29] se aprecia la interfaz de supervisor.



figura[29]. interfaz de supervisor.

IV. CONCLUSIONES

A lo largo del desarrollo del presente proyecto, se evidenció la importancia de integrar la programación orientada a objetos, la programación modular y el uso de interfaces gráficas modernas para la construcción de un sistema funcional de gestión contractual. La implementación de los módulos auth.py y supervision.py permitió separar la lógica de negocio del servidor, mientras que los archivos root1.py, root2.py y root3.py materializaron una interfaz gráfica intuitiva y diferenciada por roles, desarrollada con CustomTkinter.

La comunicación full-stack entre el front-end y el servidor, realizada mediante peticiones HTTP gestionadas por supervision_client.py, garantizó la validación de datos, el control de sesiones y la persistencia en archivos JSON. De esta forma, cada operación del usuario se tradujo en llamadas a los endpoints del servidor, el cual validó credenciales, permisos e integridad de los datos antes de persistirlos.

Finalmente, se concluye que el equipo de trabajo logró aplicar exitosamente los conceptos teóricos adquiridos, obteniendo un sistema robusto, escalable y visualmente coherente con los mockups planteados inicialmente.

V. BIBLIOGRAFÍA

[1] Python Software Foundation, "json — JSON encoder and decoder," Python 3 Documentation, 2025. [En línea]. Disponible en: <https://docs.python.org/3/library/json.html>

[2] T. Schimansky, "CustomTkinter: A modern and customizable python UI-library based on Tkinter," GitHub, 2023. [Online]. Available: <https://github.com/TomSchimansky/CustomTkinter>. [Accessed: 19-May-2025].

[3] Python Software Foundation, "csv — CSV File Reading and Writing," Python 3 Documentation, 2025. [En línea]. Disponible en: <https://docs.python.org/3/library/csv.html>

[4] Python Software Foundation, "unittest — Unit testing framework," Python 3 Documentation, 2025. [En línea]. Disponible en: <https://docs.python.org/3/library/unittest.html>

[5] ngrok, "ngrok documentation - Static Domains," ngrok Inc., 2025. [Online]. Available: <https://ngrok.com/docs/domains/>. [Accessed: 19-May-2025].

